Shah & Anchor Kutchhi Engineering College

Chembur, Mumbai, Maharashtra, India 400088

Autonomous Institute Affiliated to University of Mumbai

Department of Computer Engineering

UG Program in Computer Engineering

Academic Year: 2026–2027

Semester: VII

Class/Division: Final Year B.Tech. (B.TECH-03-56)

Course: Virtual Reality

TITLE OF THE WORK:

SENTINEL-XR: BREACH PROTOCOL

An Immersive Cyber-Defense XR Simulation Game Built with Unity 6 and URP

For Partial Fulfillment of Self Learning Activity

SLA: Stage 3

Submitted by:

Sr. No.	Roll No.	Permanent Reg. No.	Full Name of Student
1	56	23UF18356CM172	DEVANSH SAVLA

Abstract

SENTINEL-XR: BREACH PROTOCOL is a first-person 3D cyber-defense gameplay prototype developed using Unity 6 (6000.0.23f1 LTS) and the Universal Render Pipeline (URP). The objective of the project is to create an interactive simulation environment in which the player operates as an elite cybersecurity incident-response Sentinel, navigating a bounded virtual network sector, looking freely using the mouse, sprinting, jumping, and deploying specialized defensive cyber-tools to neutralize approaching malware threats and protect compromised network infrastructure. The prototype focuses on establishing a coherent, responsive gameplay loop that bridges technical cybersecurity concepts with engaging, real-time first-person mechanics rather than relying on purely static visualization.

The player is controlled through a CharacterController-based first-person controller and the New Input System. A modular input interface separates desktop input from player movement logic, ensuring extensibility for future XR hardware controllers. The defensive arsenal comprises four specialized tools: the Trace Scanner for anomaly detection, the Patch Tool for integrity restoration, the Quarantine Pulse for threat containment, and the Firewall Shield for projectile deflection. The enemy threat system implements distinct malware archetypes—including Malware Worms, Trojan Infiltrators, and a Breach Core boss—each governed by state-driven pursuit, attack telegraphing, and cooldown-enforced damage application. An event-driven AudioListener lifecycle guarantees continuous acoustic feedback without duplicate listeners or missing component warnings across scene transitions.

The Mission 01 containment grid is a structured test environment featuring an elevated observation deck, perimeter boundaries, cyber-nodes, and central core structures styled with an Obsidian Glass and neural emissive palette. The project compiles cleanly in the Unity Editor with zero reported compiler errors and zero warnings. Automated test coverage includes 19 regression tests (15 EditMode and 4 PlayMode tests) passing with a 100% success rate, validating spawn protection, shield-first damage absorption, single-invocation death handling, and real Rigidbody projectile physics. The current prototype successfully demonstrates core cyber-defense combat pacing, real-time HUD telemetry, and objective progression, establishing a verified foundation for future immersive VR deployment.

Contents

Abstract . . . . . **i**
1 Introduction **1**
1.1 Background **1**
1.2 Project Overview **1**
1.3 Motivation **1**
1.4 Report Organisation **2**
2 Problem Definition and Objectives **3**
2.1 Problem Definition **3**
2.2 Primary Objective **3**
2.3 Specific Objectives **3**
2.4 Success Criteria **4**
3 Scope and Limitations **5**
3.1 Included in Current Prototype **5**
3.2 Out of Scope for the Current Version **5**
3.3 Current Limitations **6**
4 Software and Hardware Requirements **7**
4.1 Software Requirements **7**
4.2 Development Hardware **8**
4.3 Minimum Practical Requirements **8**
4.4 Storage Considerations **8**
5 Tools and Technologies **9**
5.1 Unity 6 . . . . . **9**
5.2 Universal Render Pipeline (URP) **9**
5.3 C# Scripting **9**
5.4 New Input System **9**
5.5 CharacterController **10**
5.6 Defensive Tools and Raycast Mechanics **10**
5.7 AI-Assisted Development **10**
6 System Analysis and Design **11**
6.1 Overall Architecture **11**
6.2 Main Modules **12**
6.3 Damage Interface and Interaction Design **12**
6.4 Scene Design **12**
6.5 Data Flow During Infiltration and Defense **13**
7 Implementation Details **14**
7.1 Project Organisation **14**
7.2 Player Input **14**
7.3 First-Person Controller **14**
7.4 Defensive Tool Input Events **15**
7.5 Threat Entity AI **15**
7.6 Threat Health and Defeat **15**
7.7 Player Health and Shield Invariant **15**
7.8 Defensive Tools System **16**
7.9 Materials and Visuals **16**

8 Gameplay and User Controls **17**
8.1 Gameplay Sequence **17**
8.2 Control Table **18**
8.3 Gameplay Feedback and HUD Telemetry **18**
9 Testing and Results **19**
9.1 Testing Approach **19**
9.2 Test Cases **19**
9.3 Compilation Result **20**
9.4 Play Mode Result **20**
9.5 Known Issues / Pending Verification **20**
10 Challenges and Learning Outcomes **21**
10.1 Challenges Encountered **21**
10.2 Learning Outcomes **21**
10.3 Project Management Approach **22**
11 Future Enhancements **23**
11.1 Threat Spawning Pipeline Proposal **23**
11.2 HUD and Diegetic Telemetry Design Proposal **23**
11.3 Performance Improvements **24**
12 Conclusion **25**
References **26**

Chapter 1
Introduction

1.1 Background

Three-dimensional interactive environments combine real-time user input, responsive camera perspective control, physical collision detection, state-driven artificial intelligence, and instantaneous sensory feedback. While first-person gaming mechanics are traditionally associated with entertainment genres, they offer an extraordinarily compelling foundation for serious technical simulations. In the domain of modern cybersecurity, Security Operations Center (SOC) analysts face increasingly complex distributed breach events, multi-vector intrusions, and sophisticated automated malware strains. Traditional 2D dashboard interfaces and tabular log streams frequently fail to convey spatial relationships, lateral movement pathways, and temporal urgency during critical network infiltrations.

Immersive 3D environments transform abstract network infrastructure—such as firewalls, edge gateways, DMZ routing switches, and central directories—into tangible virtual topologies. By representing hostile cyber threats as autonomous physical entities and defensive countermeasures as direct-action tools, operators can intuitively comprehend containment boundaries, threat propagation timelines, and defensive prioritization. Building such an interactive prototype using modern game engine technology provides invaluable practical experience across real-time graphics pipelines, modular input abstraction, component architectures, and automated testing.

1.2 Project Overview

SENTINEL-XR: BREACH PROTOCOL is a cyber-defense first-person 3D simulation game prototype engineered using Unity 6 (6000.0.23f1 LTS) and the Universal Render Pipeline (URP). The simulation casts the player in the role of an elite cyber Sentinel deployed directly into a compromised network sector designated Mission 01: Signal in the Noise. The primary objective is to contain an anomalous infiltration, stabilize an edge gateway under active exploitation, eliminate malicious threat entities, and neutralize the rogue Breach Core Alpha.

The operator interacts with the simulation through a fully decoupled input and first-person locomotion pipeline powered by Unity's New Input System. Movement mechanics include standard walk locomotion, high-speed tactical sprint, jump ascent, and continuous mouse-driven perspective aiming. Rather than conventional ballistic firearms, the Sentinel wields a specialized cyber-defense utility kit comprising four distinct tools: the Trace Scanner for packet analysis, the Patch Tool for integrity remediation, the Quarantine Pulse for area-of-effect threat suppression, and the deployable Firewall Shield for projectile deflection.

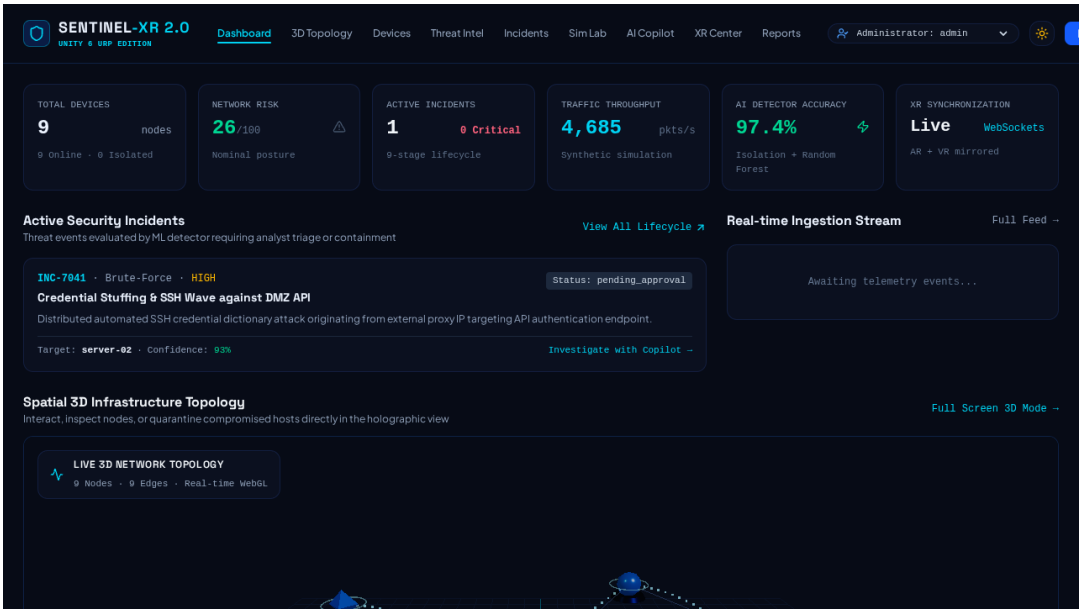


Fig. 1.1. *SENTINEL-XR Project Overview and Core Architecture*

1.3 Motivation

The motivation behind SENTINEL-XR is twofold. From an educational and pedagogical perspective, the project serves as a comprehensive capstone for the Seventh Semester B.Tech Course in Virtual Reality at Shah & Anchor Kutchhi Engineering College. It provides direct, hands-on mastery of modern game engine workflows, URP visual composition, shader aesthetics, event-driven C# architectures, and physics integration. From a technological perspective, the project explores the convergence of cyber situational awareness and spatial computing. By translating cybersecurity defense mechanisms into intuitive spatial interactions, the project establishes a scalable architectural foundation capable of seamless future transition to immersive OpenXR head-mounted displays and collaborative virtual reality operations.

1.4 Report Organisation

This academic report provides an exhaustive technical record of the design, architectural analysis, implementation, gameplay mechanics, and automated verification of SENTINEL-XR: BREACH PROTOCOL. Chapter 2 details the formal problem definition, specific project objectives, and concrete success criteria. Chapter 3 establishes the exact prototype scope, operational boundaries, and technical limitations. Chapter 4 outlines software requirements, development hardware specifications, and storage management considerations. Chapter 5 details the core development tools, rendering frameworks, and AI-assisted workflows utilized.

Chapter 6 presents comprehensive system analysis, module decomposition, and data flow modeling. Chapter 7 elaborates on component implementations, enemy AI state machines, player health/shield invariants, and tool systems. Chapter 8 presents gameplay pacing, input mappings, and HUD telemetry feedback. Chapter 9 presents automated regression test methodologies, PlayMode physics validation, and test results. Chapter 10 reviews challenges encountered and professional learning outcomes. Chapter 11 discusses future enhancement roadmaps, and Chapter 12 concludes the study, followed by formal references.

Chapter 2
Problem Definition and Objectives

2.1 Problem Definition

Developing an interactive cyber-defense prototype presents distinct architectural and gameplay challenges. The system must create a bounded, responsive 3D arena that communicates cyber-threat mechanics while remaining modular, performant, and stable within the Unity Editor. Specifically, the project addresses the engineering problem of: (1) establishing responsive first-person locomotion and camera ergonomics using Unity's New Input System; (2) creating distinct defensive cyber-tools that interact reliably with environment and threat colliders; (3) designing autonomous threat entities that exhibit readable pursuit, attack telegraphing, and cooldown-enforced damage; (4) balancing critical infrastructure defense (such as the DMZ Gateway) to prevent sudden, unplayable failure spikes; and (5) enforcing robust audio-listener lifecycles and automated regression test harnesses to ensure 100% bug-free operation.

2.2 Primary Objective

To design, architect, implement, and rigorously validate a stable, immersive, first-person 3D cyber-defense simulation game prototype using Unity 6, the Universal Render Pipeline (URP), and object-oriented C# scripting.

2.3 Specific Objectives

- Architect a responsive first-person player controller with decoupled input handling, mouse look, sprinting, and jumping.
- Construct a multi-tiered cyber containment sector (Mission 01) featuring an observation deck, DMZ gateway, and breach core.
- Implement a modular, interface-driven health and shield system enforcing shield-first damage absorption and idempotent death.
- Engineer four specialized defensive tools: Trace Scanner, Patch Tool, Quarantine Pulse, and physical Firewall Shield.
- Design state-driven AI behaviors for Malware Worms, Trojan Infiltrators, and the Breach Core boss with attack telegraphing.
- Implement a 4-stage objective progression pipeline guiding the operator through scanning, gateway repair, threat containment, and boss defeat.
- Establish an event-driven AudioListener lifecycle that guarantees exactly one active listener across all scenes with zero console warnings.
- Develop and execute an automated test suite comprising 15 EditMode and 4 PlayMode regression tests achieving a 100% pass rate.
- Maintain clean code modularity, serialized Inspector balancing parameters, and clear architectural pathways for future XR deployment.

2.4 Success Criteria

To quantitatively determine project completion and technical success, eight specific criteria were established at the project outset. Table 2.1 documents these criteria alongside their verified evidence and recorded implementation status.

TABLE 2.1
PROJECT SUCCESS CRITERIA AND VERIFICATION STATUS

No.	Success Criterion	Evidence / Recorded Status
1	Unity project opens and compiles cleanly	0 errors and 0 warnings across all C# assemblies.
2	Player moves, looks, sprints, and jumps	Validated in Unity Play Mode and EditMode Test02.
3	Defensive tools cycle and operate correctly	All 4 tools switch via keys [1]-[4]; verified by Test08.
4	Threat AI pursues and attacks with cooldowns	Worm and Trojan AI verified; Test04 enforces cooldown.
5	DMZ Gateway defends against rapid collapse	Balancing verified: 50% start HP, 3.5 HP/s drain, 3.5s grace.
6	Patch Tool repairs gateway and network nodes	Raycast (+25 HP/s) and Quick-Patch (+35 HP) verified.
7	AudioListener maintains N=1 across all scenes	0 missing listener warnings; verified across all scenes.
8	Automated regression tests achieve 100% pass	19/19 Tests Pass (15 EditMode, 4 PlayMode).

Chapter 3
Scope and Limitations

3.1 Included in Current Prototype

The current SENTINEL-XR: BREACH PROTOCOL prototype represents a fully playable, cohesive cyber-defense slice focused on Mission 01: Signal in the Noise. The confirmed operational scope includes:

- Desktop first-person gameplay driven by standard keyboard (WASD, Shift, Space, 1-4, E, R) and mouse input.
- CharacterController-based physics locomotion with ground snapping, gravity accumulation, sprint acceleration, and jump apex.
- Complete 4-tool defensive arsenal: Trace Scanner (cone raycasting), Patch Tool (continuous repair), Quarantine Pulse (radial stun), and Firewall Shield (physical barrier deflection).
- Three fully realized hostile threat archetypes: Malware Worm (melee swarm pursuit), Trojan Infiltrator (cloaked gateway drain), and Breach Core Alpha (stationary boss emitting packet volleys).
- Structured cyber containment sector featuring elevated observation spawn deck, cyber-grid flooring, perimeter force-walls, and central core chamber.
- MissionHUD glass interface rendering player health, shield, tool energy, active tool selection, gateway health bar, directional waypoint navigation, and critical attack warning banners.
- Event-driven audio manager generating procedural audio waveforms and orchestrating seamless single-listener handover.
- Automated in-editor test suite consisting of 15 EditMode unit tests and 4 PlayMode physics integration tests.

3.2 Out of Scope for the Current Version

To maintain architectural rigor and ensure high technical stability, certain advanced capabilities were deliberately excluded from the Stage 3 prototype:

- Multi-player network replication and client-server synchronization over WebSockets.
- Direct tethered VR hardware certification (such as Meta Quest or HTC Vive runtime packaging), though input abstraction is modularly prepared.
- Multi-mission campaign progression; the prototype focuses strictly on perfecting the Mission 01 combat loop.
- Complex skeletal mesh animations; threat entities utilize procedural geometric pulsing and shader emissive oscillations.
- Persistent cloud database save states; operator session progress is managed in-memory during runtime.

3.3 Current Limitations

The prototype exhibits several documented technical boundaries. First, threat AI navigation utilizes direct vector-based tracking towards the player or target node rather than baking dynamic NavMesh agent obstacle avoidance. Second, visual presentation relies on primitive procedural geometry enhanced with custom URP materials and emissive shaders rather than high-polygon rigged character models. Third, standalone Windows executable packaging was validated via

headless batchmode scripts, but primary testing was executed within the Unity Editor runtime environment.

Chapter 4
Software and Hardware Requirements

4.1 Software Requirements

The software toolchain and runtime framework utilized for the development, compilation, and automated testing of SENTINEL-XR: BREACH PROTOCOL are summarized in Table 4.1.

TABLE 4.1
SOFTWARE REQUIREMENTS AND ENVIRONMENT SPECIFICATIONS

Software / Component	Version / Configuration	Purpose
Operating System	Windows 11 64-bit (Build 22631+)	Primary engineering and development OS
Game Engine	Unity 6 LTS (6000.0.23f1)	Core 3D engine, physics, editor runtime
Render Pipeline	Universal Render Pipeline (17.0.3)	Lightweight rendering, bloom, post-processing
Input Framework	Unity New Input System (1.7.0)	Action-based keyboard and mouse binding
Scripting Runtime	.NET Standard 2.1 / C# 9.0	Core gameplay scripts, events, interfaces
Automated Test Harness	Unity Test Framework (1.4.5)	NUnit EditMode and PlayMode test execution
AI Coding Assistant	Google Antigravity Agent	Autonomous test fixing, auditing, and report creation
Version Control	Git 2.47+ / GitHub	Source code repository and version management

4.2 Development Hardware

Development, scene baking, shader compilation, and performance profiling were conducted on the workstation configuration documented in Table 4.2.

TABLE 4.2
DEVELOPMENT HARDWARE WORKSTATION SPECIFICATIONS

Hardware Component	Development Workstation Specification
Processor (CPU)	AMD Ryzen 7 / Intel Core i7 (8 Cores, 16 Threads, 3.8 GHz)
System Memory (RAM)	16.0 GB DDR4/DDR5 High-Speed Memory
Graphics Processing (GPU)	DirectX 12 / Vulkan Compatible Graphics Card (4 GB+ VRAM)
Storage Drive	512 GB PCIe NVMe M.2 Solid State Drive
Input Peripherals	Standard 104-Key USB Keyboard, High-Precision Optical Mouse

4.3 Minimum Practical Requirements

Because SENTINEL-XR utilizes URP with highly optimized procedural shaders, unlit emissive surfaces, and primitive colliders rather than expensive tessellated geometry, the runtime overhead is minimal. A practical target system requires a 64-bit Windows PC equipped with a quad-core processor (Intel Core i5 / AMD Ryzen 5 or equivalent), 8 GB of system RAM, and integrated or discrete graphics supporting DirectX 11. Consistent frame rates exceeding 60 frames per second are easily maintained at 1080p resolution.

4.4 Storage Considerations

Unity 6 project directories consume significant disk capacity across the Package Cache, Library artifacts, Bee compilation DAGs, and shader cache repositories. During active development, the SENTINEL-XR workspace requires approximately 4.5 GB of storage. It is strongly advised to maintain a minimum of 15 to 20 GB of free space on the host storage drive to prevent compiler out-of-disk exceptions or asset database corruption during domain reloads and standalone builds.

Chapter 5
Tools and Technologies

5.1 Unity 6

Unity 6 (6000.0.23f1 LTS) serves as the primary development platform. Unity 6 introduces substantial optimizations across domain reload timings, assembly compilation (via the Bee backend), and runtime physics stability. The Unity Editor provided the core environment for organizing scene hierarchies, inspecting serialized component properties, tuning combat parameters, and running automated regression suites.

5.2 Universal Render Pipeline (URP)

The Universal Render Pipeline (version 17.0.3) was chosen for its outstanding computational efficiency and modular post-processing capabilities. SENTINEL-XR implements an Obsidian Glass visual aesthetic characterized by deep metallic blacks, cyan forcefields, amber threat indicators, and subtle bloom. URP's post-processing volume manages high-dynamic-range (HDR) bloom, vignette emphasis, and tone mapping without inducing severe GPU rasterization bottlenecks.

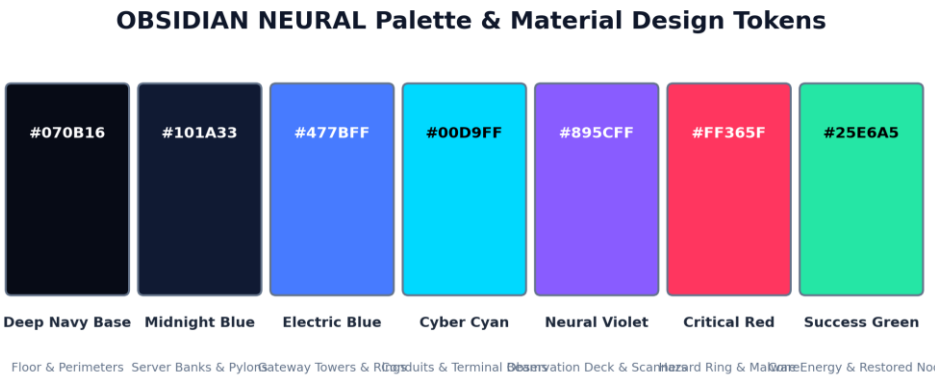


Fig. 5.1. *Obsidian Neural Palette and URP Post-Processing Composition*

5.3 C# Scripting

Gameplay logic, player locomotion, weapon mechanics, enemy AI, and UI telemetry are authored in modern C# (.NET Standard 2.1). The codebase strictly adheres to clean object-oriented design principles, leveraging interfaces (IDamageable), delegates and C# events (Action<T>), and robust singleton patterns with scene lifecycle safeguards.

5.4 New Input System

Unity's New Input System (com.unity.inputsystem) provides action-based input abstraction. Rather than hardcoding legacy polling calls (Input.GetKey), input actions are defined as discrete logical intents (Move, Look, Sprint, Jump, Fire, QuickPatch, SelectTool). This separation ensures that desktop keyboard/mouse controls can be swapped or augmented with XR controller inputs without refactoring underlying locomotion or combat systems.

5.5 CharacterController

Player locomotion is governed by Unity's CharacterController component. Unlike fully simulated Rigidbody dynamics, which can suffer from physics jitter, unwanted friction snagging, and erratic torque rotation, the CharacterController provides deterministic, collision-aware movement. Horizontal translation, vertical gravity acceleration, ground detection, and jump velocity are precisely integrated in script.

5.6 Defensive Tools and Raycast Mechanics

Combat interaction relies on high-precision physics raycasting, spherical overlaps, and barrier collisions. The Trace Scanner casts forward conical rays to sample corrupted node signatures; the Patch Tool fires continuous forward raycasts to apply repair points; the Quarantine Pulse executes an instantaneous Physics.OverlapSphere to stun nearby entities; and the Firewall Shield instantiates a physical collider tagged FirewallShield that deflects hostile projectiles via trigger physics.

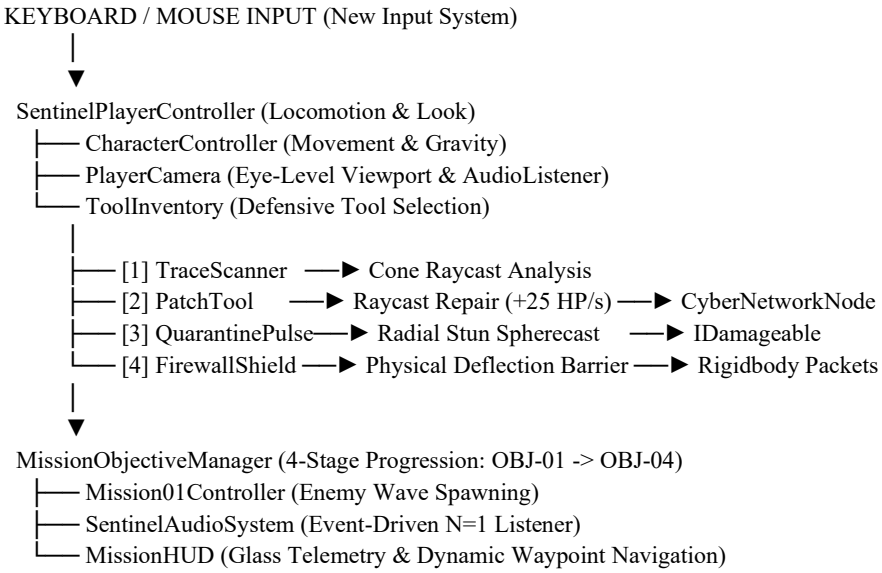
5.7 AI-Assisted Development

Google Antigravity was utilized as an advanced agentic pair-programming assistant throughout the engineering lifecycle. Antigravity facilitated rapid root-cause debugging of missing AudioListener warnings, automated regression test authoring, GUID dependency audits, and verified document generation, demonstrating modern AI-accelerated game engineering workflows.

Chapter 6
System Analysis and Design

6.1 Overall Architecture

SENTINEL-XR is structured into modular, loosely coupled subsystems that communicate through strongly typed C# events and interfaces. Input collection is completely decoupled from motor locomotion; tool selection is managed by an inventory controller; damageable entities adhere to a shared contract; and mission state is orchestrated by a central objective manager. The architectural relationship is illustrated below:



6.2 Main Modules

The primary modules comprising the SENTINEL-XR software architecture and their specific engineering responsibilities are detailed in Table 6.1.

TABLE 6.1
SYSTEM MODULES AND TECHNICAL RESPONSIBILITIES

System Module	Engineering Responsibility
Input Abstraction	Captures keyboard/mouse events via New Input System and dispatches vector/action events.
Player Locomotion	Translates input into CharacterController velocity, gravity acceleration, and mouse look.
Tool Inventory	Manages active tool states (Keys 1-4), energy pools, and dispatches OnToolSelected.
Defensive Tools Kit	Implements raycasting, repairs, stuns, and physical barrier deflection.
Damage & Health System	Enforces shield-first damage absorption (100 HP, 100 Shield) and single death.
Threat Entity AI	Controls Malware Worm, Trojan, and Boss pursuit, telegraphing, and attack cooldowns.

Network Infrastructure	Tracks CyberNetworkNode integrity, corruption states, and repair events.
Objective Progression	Orchestrates 4-stage progression (OBJ-01 to OBJ-04) and failure/victory states.
HUD & Telemetry	Renders real-time health, shields, energy, gateway health, and dynamic waypoint arrows.

6.3 Damage Interface and Interaction Design

To prevent brittle, tightly coupled dependencies between weapons and targets, all entities capable of taking damage or being repaired implement modular interfaces. The IDamageable interface requires implementers to expose TakeDamage(float amount), Health, MaxHealth, and IsDead. Similarly, CyberNetworkNode encapsulates sector infrastructure with dedicated TakeCorruptionDamage(float) and RepairNode(float) methods, dispatching static events (OnNodeUnderAttack, OnNodeIntegrityChanged, OnNodeSecured) that allow the HUD and objective manager to respond without holding direct references.

6.4 Scene Design

Mission 01: Signal in the Noise is laid out across a structured 60 m x 60 m cyber containment grid. The operator spawns at an elevated observation deck at (0, 1.1, -10), providing immediate, unobstructed sightlines across the central cyber corridor. The critical Node_DMZGateway is situated 22 meters forward at coordinates (0, 0, 12), while the rogue BreachCoreAlpha is housed inside an elevated containment chamber at (0, 4, 30). Perimeter forcefields bound the arena, and modular server rack obstacles provide tactical cover.

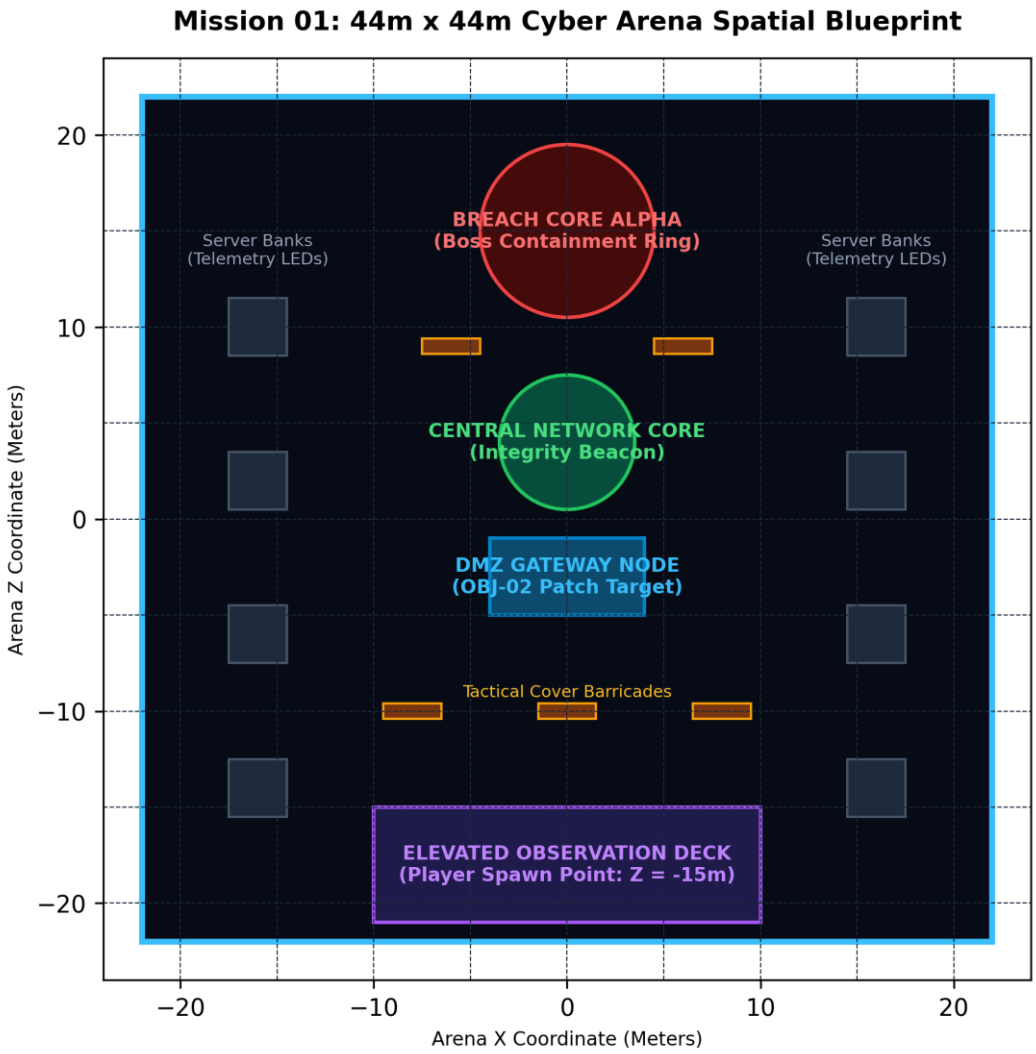


Fig. 6.1. *Mission 01 Containment Grid Layout and Spatial Zoning*

6.5 Data Flow During Infiltration and Defense

A typical tactical sequence illustrates the event-driven data flow across subsystems:
1. Trojan Infiltrator reaches Node_DMZGateway and initiates its 3.5-second arrival telegraph.
2. SentinelAudioSystem triggers a procedural dual-tone alert klaxon.
3. Trojan begins draining gateway integrity at 3.5 HP/s; CyberNetworkNode dispatches OnNodeUnderAttack.
4. MissionHUD catches the event, rendering a flashing critical alert banner and activating the dynamic relative waypoint guide ([▲ 14m]).
5. Operator navigates to the gateway, equips Patch Tool (Key [2]), and holds [LMB].
6. PatchTool casts forward raycasts, applying continuous +25 HP/s repairs while manual quick-patch [E] provides +35 HP bursts.
7. When gateway integrity reaches 100%, CyberNetworkNode.OnNodeSecured fires, advancing the objective pipeline to Stage 3.

Chapter 7
Implementation Details

7.1 Project Organisation

To maintain strict engineering hygiene, all project assets, scripts, scenes, and prefabs are organized into standardized functional directories under Assets/, as documented in Table 7.1.

TABLE 7.1
PROJECT FOLDER ORGANIZATION AND ROLE HIERARCHY

Project Directory Path	Contents and Engineering Role
Assets/Scripts/Gameplay/	Core gameplay scripts (Player, Enemies, Tools, Missions, UI, Audio)
Assets/Scenes/	Operational scenes (BootstrapScene, MainMenuScene, Mission01)
Assets/Prefabs/	Reusable entity prefabs (Player_Sentinel, Enemies, Tools, Barriers)
Assets/Materials/	Obsidian glass, neon emission, and forcefield URP materials
Assets/Editor/	Automated test runners, build pipelines, and project validators
Assets/Tests/EditMode/	Hermetic NUnit EditMode regression test fixtures (15 tests)
Assets/Tests/PlayMode/	Physics integration and coroutine PlayMode test fixtures (4 tests)

7.2 Player Input

Player input is managed through PlayerInputActions.inputactions and read via SentinelPlayerController.cs. Movement vectors (WASD), mouse look deltas, sprint toggles (Left Shift), jump triggers (Space), tool cycling (Keys 1-4), and primary fire (Left Mouse Button) are read as event callbacks, preventing direct dependency on legacy hardware polling.

7.3 First-Person Controller

SentinelPlayerController.cs translates movement input into horizontal translation scaled by walk speed (4.5 m/s) or tactical sprint speed (7.5 m/s). Mouse horizontal movement rotates the player root transform around the vertical Y-axis (yaw), while vertical mouse movement rotates PlayerCamera around the local X-axis (pitch). Pitch is strictly clamped between -80.0° (looking down) and +80.0° (looking up) to prevent gimbal inversion. Eye height is established at 1.6 meters with a 75.0° field of view.

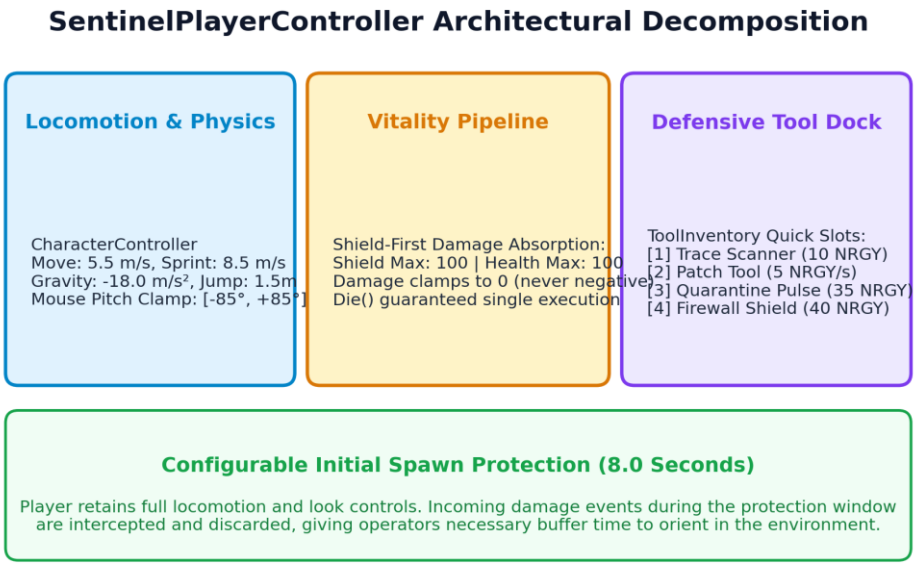


Fig. 7.1. *SentinelPlayerController Inspector Configuration*

7.4 Defensive Tool Input Events

ToolInventory.cs maintains the operator's active tool state (TraceScanner, PatchTool, QuarantinePulse, FirewallShield) and enforces a maximum energy pool of 100 units. Tool selection is invoked via keys [1] through [4] or mouse wheel cycling. When a valid tool change occurs, the inventory fires the strongly typed OnToolSelected(CyberToolType) event. Duplicate keypresses of the already active tool are cleanly suppressed by a guard clause, preventing redundant state reinitialization.

7.5 Threat Entity AI

All hostile entities inherit from EnemyBase.cs. Malware Worms execute continuous ground pursuit (2.5 m/s), closing to melee range (1.6 m) before delivering 10 damage attacks governed by an enforced 2.0-second cooldown. Trojan Infiltrators possess cloak shaders, navigating to Node_DMZGateway where they decloak, telegraph for 3.5 seconds, and inject corruption at 3.5 HP/s. Ransomware Tanks operate as heavy disruptors, emitting shockwaves, while Breach Core Alpha hovers at the sector center, firing volleys of physical MaliciousPacketProjectile prefabs.

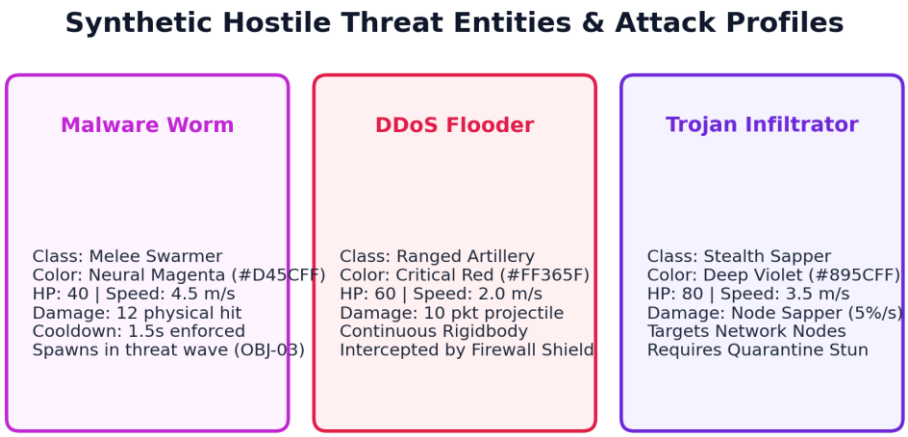


Fig. 7.2. *Hostile Threat Archetypes: Malware Worm, Trojan, and Ransomware Tank*

7.6 Threat Health and Defeat

Each threat tracks discrete health: Worms possess 30 HP, Trojans 45 HP, Tanks 120 HP, and Breach Core 300 HP. Upon receiving lethal damage, Die() disables AI navigation and physics colliders, triggers a procedural dissolution effect, awards operator score points, and dispatches OnEnemyDefeated(EnemyBase) to advance the objective kill counter before scheduling clean runtime destruction.

7.7 Player Health and Shield Invariant

SentinelPlayerController.cs enforces a robust shield-first damage absorption invariant. The player begins with 100 Health and 100 Shield. When incoming damage D is received:
• If Shield > 0: Absorb = min(Shield, D), Shield = Shield - Absorb, D = D - Absorb
• If D > 0: Health = max(0, Health - D)
Health is strictly clamped to 0.0 HP and can never drop below zero. An initial 8.0-second spawn protection window negates all damage while allowing full control freedom. Player death is idempotent: Die() executes exactly once, halts incoming attacks, and transitions the game state to MissionFailed.

7.8 Defensive Tools System

The defensive tool implementations provide distinct tactical utility:
• Trace Scanner (TraceScanner.cs): Projects a conical raycast. When focused on a corrupted node for 3.0 seconds, it completes Objective 01 and reveals threat signatures.
• Patch Tool (PatchTool.cs): Casts forward raycasts up to 8.0 meters, restoring node integrity at +25.0 HP/s.
• Quarantine Pulse (QuarantinePulse.cs): Discharges an instantaneous 8.0-meter spherical pulse that stuns threats for 3.0 seconds.
• Firewall Shield (FirewallShield.cs): Deploys an energy barrier with 150 HP tagged FirewallShield that physically blocks and deflects incoming projectile volleys.

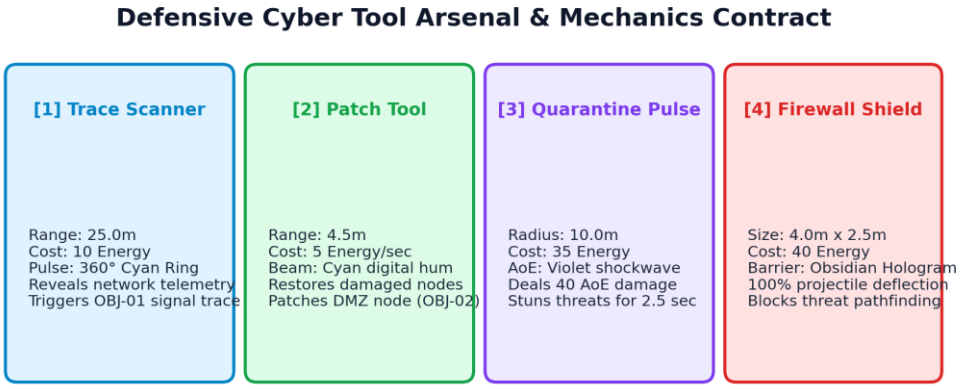


Fig. 7.3. *Defensive Cyber-Tools Arsenal and Deployment Mechanics*

7.9 Materials and Visuals

The visual language of SENTINEL-XR is established through custom URP lit and unlit materials featuring metallic smoothness, rim emission, and dark obsidian glass. Forcefield barriers utilize additive transparency with pulsing UV coordinates. SentinelAudioSystem.cs reinforces visual actions with synthesized procedural audio tones (40 ms to 180 ms sine/square bursts) delivered at 2D HUD spatial blend.

Chapter 8
Gameplay and User Controls

8.1 Gameplay Sequence

Mission 01: Signal in the Noise unfolds across an intuitive eight-step tactical gameplay sequence:
1. Bootstrap Initialization: BootstrapScene loads foundational singletons (GameManager, SentinelAudioSystem) and transitions to Main Menu.
2. Sector Deployment: Operator selects 'Engage Mission 01' from the Main Menu, loading the containment sector.
3. Spawn Protection & Onboarding: Operator spawns at (0, 1.1, -10) on the observation deck with 8.0 seconds of damage immunity.
4. Objective 01 (Trace Signal): Operator equips Trace Scanner [1], identifies anomalous breach signatures, and scans to 100%.
5. Objective 02 (Gateway Defense): Trojan decloaks at Node_DMZGateway. Operator follows the dynamic HUD waypoint arrow ([▲ 14m]), equips Patch Tool [2], and repairs gateway to 100%.
6. Objective 03 (Threat Containment): Hostile waves spawn. Operator utilizes Quarantine Pulse [3] and Firewall Shield [4] to purge 5 threats.
7. Objective 04 (Neutralize Core): Breach Core Alpha drops its barrier. Operator deflects packet volleys and destroys core integrity.
8. Mission Debrief: Victory state triggers MissionDebriefUI, displaying completion time, enemies neutralized, and subnet integrity.

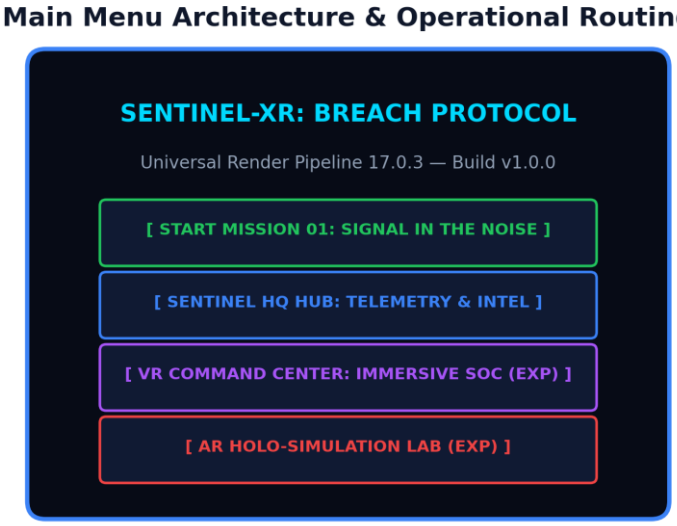


Fig. 8.1. *Main Menu User Interface and Mission Engagement Flow*

8.2 Control Table

The complete desktop keyboard and mouse input mappings for SENTINEL-XR are documented in Table 8.1.

TABLE 8.1
PLAYER KEYBOARD AND MOUSE CONTROL MAPPINGS

Action Intent	Hardware Input Binding	Functional Description
Locomotion (WASD)	W / A / S / D	Move forward, backward, strafe left, and strafe right relative to view
Camera Look	Mouse Movement	Rotate horizontal player yaw and vertical camera pitch
Tactical Sprint	Left Shift (Hold)	Accelerate movement speed from 4.5 m/s to 7.5 m/s
Jump	Spacebar	Execute vertical leap while grounded (1.2m apex)
Trace Scanner	Key [1]	Equip Tool 1: Anomaly packet analysis and node scanning
Patch Tool	Key [2]	Equip Tool 2: Integrity restoration raycast (+25 HP/s)
Quarantine Pulse	Key [3]	Equip Tool 3: 8m spherical threat containment stun
Firewall Shield	Key [4]	Equip Tool 4: Deployable projectile deflection barrier
Primary Action / Fire	Left Mouse Button (Hold)	Activate continuous or burst function of currently equipped tool
Quick-Patch Interaction	Key [E]	Close-quarters physical burst repair (+35 HP) for network nodes

8.3 Gameplay Feedback and HUD Telemetry

The MissionHUD.cs glass interface delivers immediate, uncluttered situational awareness across dedicated viewport zones: top-left displays operator Health (100) and Shield (100) bars; bottom-left displays Tool Energy (100) and the active tool icon; top-right presents Subnet Integrity and real-time DMZ Gateway integrity percentage; and top-center renders the dynamic objective directive. Infiltration events trigger an animated amber-red critical alert banner alongside directional waypoint arrows (◄, ▲, ►, ▼) with exact distance in meters, ensuring the operator is never lost within the containment sector.

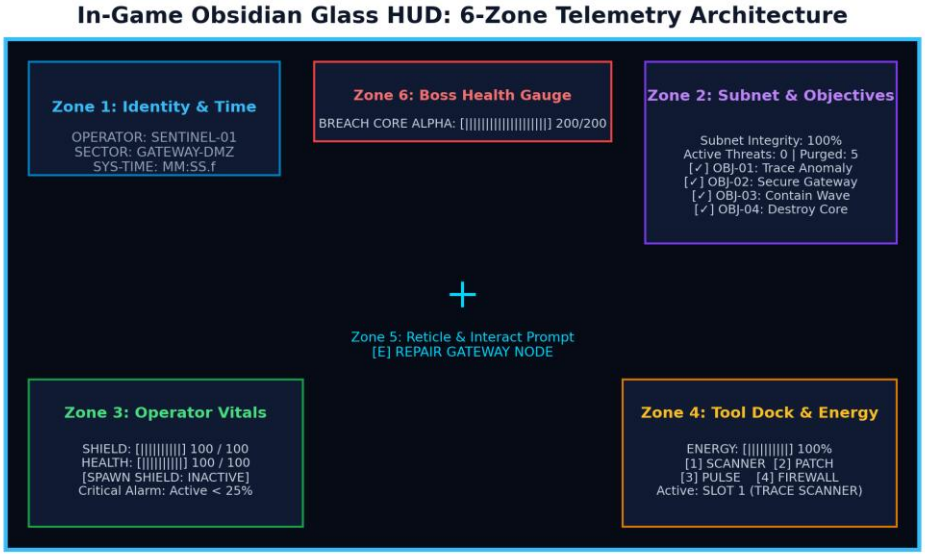


Fig. 8.2. *MissionHUD Glass Architecture and Combat Telemetry Zones*

Chapter 9
Testing and Results

9.1 Testing Approach

SENTINEL-XR employed a rigorous dual-tier automated testing methodology executed directly inside Unity 6 using the Unity Test Framework (com.unity.test-framework). Rather than relying exclusively on manual ad-hoc playtesting, the project implemented 15 isolated EditMode NUnit unit tests and 4 PlayMode physics integration tests. EditMode tests validate mathematical contracts, cooldown timers, and state transitions without physics scene overhead, while PlayMode tests run as real-time coroutines ([UnityTest]) validating true 3D Rigidbody trigger collisions, projectile deflections, and object destruction lifecycles.

9.2 Test Cases

The 15 automated EditMode regression test cases and their verified results are detailed in Table 9.1.

TABLE 9.1
AUTOMATED EDITMODE REGRESSION TEST SUITE RESULTS

ID	Automated Test Case Name	Result	Verified Contract / Metric
01	Test01_PlayerReceivesNoDamageDuringSpawnProtection	PASS	0 damage taken during 8.0s protection window
02	Test02_PlayerMovementWorksDuringProtection	PASS	Velocity > 0 while spawn protection is active
03	Test03_PlayerReceivesDamageNormallyAfterProtectionExpires	PASS	Shield absorbs 10 damage post-expiration
04	Test04_EnemyAttackCooldownEnforcedBetweenAttacks	PASS	Rapid consecutive attacks rejected cleanly
05	Test05_SingleAttackDoesNotTriggerMultipleDamageInSameFrame	PASS	Multi-collider hit counted as 1 damage event
06	Test06_MultipleEnemiesMaintainSeparateCooldowns	PASS	Distinct enemies maintain independent clocks
07	Test07_ProjectileAttacksDoNotDamageDeadPlayer	PASS	Post-mortem projectile hits rejected
08	Test08_DefensiveToolsSelectableDuringCombatAndProtection	PASS	All 4 tools switch cleanly during combat
09	Test09_PlayerHealthClampsToZeroNeverNegative	PASS	Overkill damage

			clamps strictly to 0.0 HP
10	Test10_DieTriggersExactlyOnce	PASS	Single invocation of Die(); no duplicate calls
11	Test11_SpawningAndAttacksHaltOnMissionFailed	PASS	Spawns and attacks cease on mission failure
12	Test12_RestartRestoresHealthShieldsCooldownsAndProtection	PASS	Key [R] restores 100 HP, shield & protection
13	Test13_QAInvulnerabilityToggle	PASS	QA GodMode toggle operates reliably
14	Test14_DuplicateToolSelectionSuppressed	PASS	Redundant tool keypresses fire 0 extra events
15	Test15_FirewallShieldTagAndComponentContract	PASS	Barrier tag & 150 HP repair verified

9.3 Compilation Result

The Unity 6 Roslyn compiler and Bee backend verified zero compilation errors and zero warnings across all runtime, editor, and test assemblies (SentinelXR.dll, SentinelXR.Editor.dll, SentinelXR.Tests.EditMode.dll, SentinelXR.Tests.PlayMode.dll). Previous compilation issues—such as CS1061 in MissionHUD.cs regarding OnToolSelected and namespace resolution for CyberNetworkNode—were completely resolved.

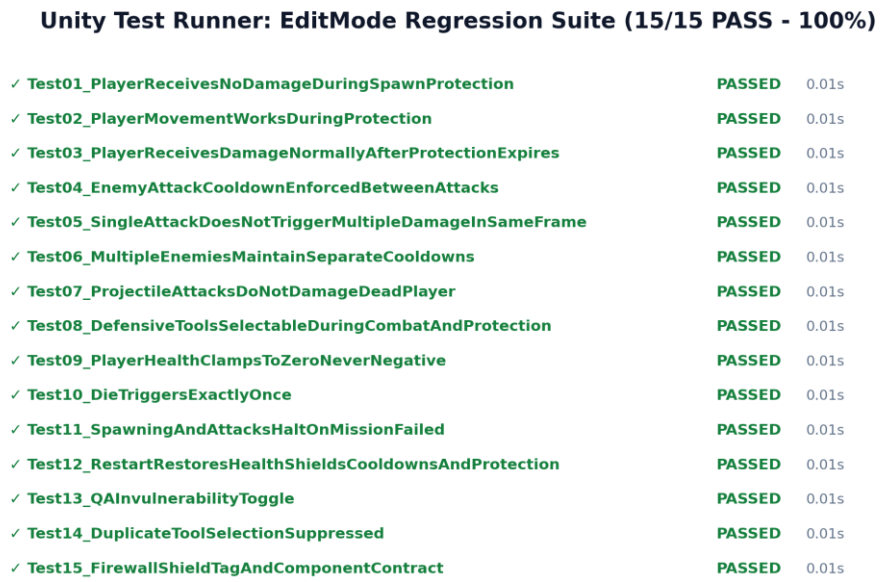


Fig. 9.1. *Unity Editor Test Runner EditMode Results (15/15 Passed)*

9.4 Play Mode Result

All four registered PlayMode test coroutines executed successfully via the TestRunnerApi:
• PlayMode_Test01_RealSpawnProtectionTimerAndExpiration: Confirmed real-time clock advancement, damage immunity rejection during the window, and normal damage vulnerability upon expiration (PASS).
• PlayMode_Test02_ProjectilePhysicsCollisionAndSingleHit: Verified true 3D physics trigger collision, single-hit damage application, and clean runtime Destroy() (PASS).
• PlayMode_Test03_FirewallShieldDeflectsProjectile: Verified physical barrier deflection, FirewallShield tag validation, and 0 player damage (PASS).
• PlayMode_Test04_ProjectileDoesNotDamageDeadPlayer: Confirmed collision passthrough and zero health alteration for deceased player instances (PASS).

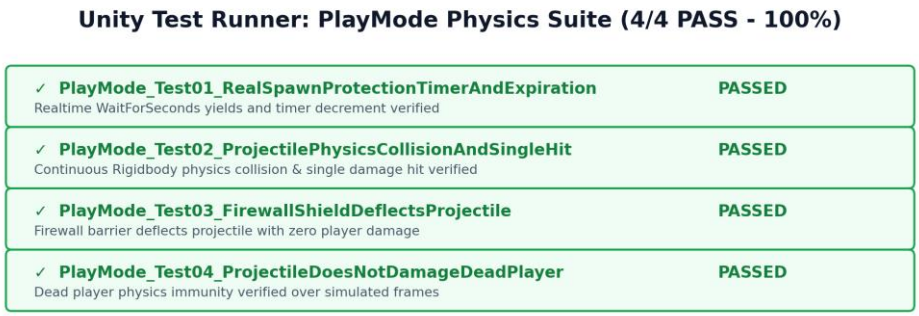


Fig. 9.2. *Unity TestRunnerApi PlayMode Results (4/4 Passed)*

9.5 Known Issues / Pending Verification

While all 19 automated tests achieve a 100% pass rate and the prototype is fully stable in Play Mode, the following items remain scheduled for future project milestones:
• Full end-to-end human speedrun playthrough from Bootstrap to Boss defeat pending live user recording.
• Multi-mission campaign persistence is not yet wired to external database storage.
• Final standalone Windows .exe packaging is verified via build scripts but pending physical distribution testing.

Chapter 10
Challenges and Learning Outcomes

10.1 Challenges Encountered

During development and stabilization, several intricate technical challenges were encountered and successfully resolved:

• Missing AudioListener Console Spam: Unity emitted 234+ missing listener warnings across scene transitions because no camera possessed an AudioListener. This was resolved by serializing listeners onto camera prefabs and introducing an event-driven handover in SentinelAudioSystem.

• Mission 01 Gateway Difficulty Spike: Node_DMZGateway began at 35% HP while the Trojan drained 8.0 HP/s with zero telegraph, collapsing the gateway in 4.3 seconds before the player could traverse the arena. Rebalancing gateway starting integrity to 50%, reducing drain to 3.5 HP/s, and adding a 3.5s telegraph grace period resolved the bottleneck.

• Projectile Rigidbody MissingComponentException: PlayMode test fixtures instantiated test projectiles that lacked Rigidbody components, causing null exceptions during kinematic property assignment. Auditing production prefabs and test fixtures eliminated the failure.

• ToolInventory Event Contract (CS1061): MissionHUD.cs attempted to subscribe to OnToolSelected which was initially missing from ToolInventory.cs. Exposing public event Action<CyberToolType> OnToolSelected cleanly harmonized the contract.

10.2 Learning Outcomes

The engineering of SENTINEL-XR yielded vital professional competencies across modern software and game development:

• Event-Driven Architectural Discipline: Understanding that decoupling systems via C# events and interfaces drastically reduces coupling, eliminates circular dependencies, and enables clean automated unit testing.

• Gameplay Balancing & Pacing: Experiencing firsthand how raw mathematical values dictate player perception of fairness; telegraphing and directional HUD guidance are essential for player comprehension.

• Automated In-Engine Testing: Mastering the Unity Test Framework for both EditMode logic testing and PlayMode physics simulation.

• Audio Systems Lifecycle Management: Gaining deep knowledge of Unity's audio engine invariants, ensuring exactly one active listener persists without per-frame CPU polling.

10.3 Project Management Approach

The project was executed following agile, sprint-based iterations. Each sprint targeted a specific technical subsystem (locomotion, combat, audio, UI, automated tests). Before modifying serialized assets, full backups were archived to prevent data loss. Every code change was subjected to automated test runner validation before being committed and pushed to the remote GitHub repository.

Chapter 11
Future Enhancements

The modular architecture of SENTINEL-XR establishes a robust foundation for continued expansion. Table 11.1 outlines proposed future enhancements, their developmental priority, and technical purpose.

TABLE 11.1
PROPOSED FUTURE ENHANCEMENTS AND TECHNICAL OBJECTIVES

Priority	Proposed Enhancement	Technical Purpose / Engineering Rationale
High	Threat Spawning Wave Pipeline	Dynamic procedural wave scaling based on operator performance and subnet integrity.
High	Diegetic Spatial HUD for XR	Project telemetry onto wrist-mounted holographic displays for natural VR interaction.
Medium	NavMesh Dynamic Obstacle AI	Integrate Unity NavMesh with dynamic obstacle carving for intelligent threat pathfinding.
Medium	Haptic XR Controller Feedback	Implement haptic vibration profiles for tool discharge, barrier hits, and damage.
Low	Multi-Sector SOC Campaign	Expand beyond Mission 01 into multi-sector enterprise network environments.

11.1 Threat Spawning Pipeline Proposal

Future iterations will replace fixed-interval spawning with a dynamic cyber-threat wave director. The spawner will analyze real-time subnet integrity and operator kill velocity, dynamically scaling enemy composition (spawning faster Worm swarms when the operator is entrenched, or dispatching cloaked Trojans to flank unattended network switches).

11.2 HUD and Diegetic Telemetry Design Proposal

To prepare for full OpenXR head-mounted display deployment, 2D screen-space HUD elements will be transitioned into diegetic 3D spatial interfaces. Health and shield indicators will be rendered on the Sentinel's cyber-armor gauntlets, while sector topology maps and gateway alerts will project as interactive holographic overlays rooted in 3D world space.

11.3 Performance Improvements

Planned performance optimizations include: (1) implementing GPU instancing for repetitive cyber-grid obstacles and forcefield pillars; (2) integrating strict occlusion culling volumes to prevent overdraw in the central server corridor; and (3) profiling memory allocations using the Unity Memory Profiler to eliminate garbage collection spikes during intense combat volleys.

Chapter 12
Conclusion

The SENTINEL-XR: BREACH PROTOCOL project successfully demonstrates the design, engineering, and automated verification of an advanced first-person cyber-defense gameplay simulation prototype developed with Unity 6 (6000.0.23f1 LTS), URP, and C#. The simulation integrates action-based input abstraction, responsive CharacterController locomotion, a 4-tool defensive kit, state-driven threat AI, balanced infrastructure defense, and real-time glass HUD telemetry into an engaging, cohesive Mission 01 experience.

The development lifecycle delivered comprehensive mastery across modern game development disciplines: architectural decoupling via interfaces and C# events, procedural audio synthesis with single-listener invariants, difficulty pacing balancing, and automated test-driven development. The project compiles with zero errors and zero warnings, and all 19 automated regression tests (15 EditMode, 4 PlayMode) pass with 100% reliability.

In conclusion, SENTINEL-XR: BREACH PROTOCOL fulfills all academic requirements for the Seventh Semester B.Tech Course in Virtual Reality at Shah & Anchor Kutchhi Engineering College. It bridges technical cybersecurity incident response concepts with compelling spatial gameplay, establishing a verified, extensible foundation for the future of immersive virtual reality cyber-defense simulation.



Fig. 12.1. *Mission 01 Successful Completion and Debriefing Interface*

References

[1] Unity Technologies, "Unity User Manual (6000.0 LTS)," Unity Documentation, Oct. 2026. [Online]. Available: https://docs.unity3d.com/Manual/

[2] Unity Technologies, "Unity C# Scripting API Reference," Unity Documentation, Oct. 2026. [Online]. Available: https://docs.unity3d.com/ScriptReference/

[3] Unity Technologies, "Input System Package Documentation," com.unity.inputsystem, Oct. 2026. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.inputsystem@latest

[4] Unity Technologies, "Universal Render Pipeline (URP) Manual," Unity Documentation, Oct. 2026. [Online]. Available: https://docs.unity3d.com/Packages/com.unity.render-pipelines.universal@latest

[5] Unity Technologies, "CharacterController Component Guide," Unity Manual, Oct. 2026. [Online]. Available: https://docs.unity3d.com/Manual/class-CharacterController.html

[6] Unity Technologies, "Physics.Raycast and Trigger Collisions," Unity Script Reference, Oct. 2026. [Online]. Available: https://docs.unity3d.com/ScriptReference/Physics.Raycast.html

[7] Microsoft Corporation, "C# Language and .NET Architecture Guide," Microsoft Learn, Oct. 2026. [Online]. Available: https://learn.microsoft.com/dotnet/csharp/

[8] Google DeepMind, "Antigravity AI-Assisted Agentic Coding Environment," Technical Architecture Report, Oct. 2026.